

C# Core Concepts & Text Adventure Porting Assignment

Theoretical Concepts Answers

1. ref vs. out

○ Explain the difference between passing arguments by ref versus out in C#. When would you choose one over the other?

- In C#, both ref and out allow you to pass arguments by reference so that a method can modify the original variable, but the main difference comes down to initialization. With ref, you **must** initialize the variable before passing it into the method. With out, you don't have to initialize the variable beforehand, but the method itself **must** assign a value to it before returning.

I actually chose to use out in our text adventure during the battle sequence: `int.TryParse(Console.ReadLine(), out int choice)`. I used out here because I didn't care what the value of choice was beforehand, I just needed the TryParse method to process the player's text input and dump the resulting integer straight into that variable.

2. The using Statement

○ What is the primary function of the using statement in C# (specifically regarding resource management/garbage collection)? Write a short pseudo-code example showing its usage.

- When talking about memory and resource management, the using statement ensures that unmanaged resources (like file streams, network connections, or database connections) are immediately and properly closed the moment a block of code finishes running. It acts as a safety net that automatically calls an object's `Dispose()` method behind the scenes, even if the game throws an exception or crashes.

For example, if we added a feature to save the player's current Health and Eloquence to a text file, I would wrap it in a using statement so the game doesn't accidentally leave the save file locked on the computer:

```
using (StreamWriter writer = new
StreamWriter("gamesave.txt"))
{
    writer.WriteLine($"HP: {player.Health}");
    writer.WriteLine($"Eloquence:
{player.Eloquence}");
} // The file is automatically closed and disposed of
right here
```

3. Nullable Variables

○ What is a nullable variable in C# (e.g., int)? Provide a code example and describe a real-world game development scenario where a nullable type is useful.

- By default, value types in C# (like int, float, or bool) cannot be null; an int will always default to 0. A nullable variable, denoted by a question mark (e.g., int?), is a special type that can hold its normal range of values plus a true null value.

In a game development scenario, this is incredibly useful for distinguishing between "zero" and "nothing." For example, if we added equippable armour to our game, we could use `int? equippedArmourDefense = null`.

If the player has no armour equipped, the value is null. If they are wearing a garbage bag they found at LaSalle, the value might be 0. Being able to differentiate between "I have an item that gives 0 defence" and "I don't have an item at all" is crucial for game logic.

4. C# Memory Management & "Foreachable" Memory

○ Explain in your own words how C# manages memory under the hood (mentioning the Garbage Collector). Specifically, define what "foreachable" memory (or the finalization queue/foreachable queue) is and how objects end up there.

- Unlike C++ where I had to manually track pointers and delete objects, C# manages memory automatically using the Garbage Collector. The GC routinely scans the "heap" (where our game objects live) to find objects that are no longer being referenced by the program, like when we defeat an enemy and no longer need their stats loaded, and frees up that memory automatically. "Foreachable" (Finalization Reachable) memory is a specific part of this process for objects that require custom cleanup via a finalizer (a destructor). When an

object is created, if it has a finalizer, C# puts a reference to it in the "Finalization Queue." When the GC realizes this object is dead (unreachable by the game), it doesn't delete it right away. Instead, it moves the reference to the "Foreachable queue." This queue tells a dedicated background thread, "Hey, run the finalization code for these objects." While it sits in this Foreachable queue, the object is technically kept alive just long enough for its cleanup code to run. Once the finalizer finishes, the object becomes truly dead, and the GC wipes its memory on the next sweep.